# Fast Failover & Resilience for Campus Networks using SDN

A plain-English explainer: the problem, what we built, and why it beats the way networks fail over today.

Software-Defined Networking · OpenFlow 1.3 · Mininet · Open vSwitch · Ryu

# 1. The problem, in everyday terms

A campus network is the set of switches and cables that connect every computer, phone, Wi-Fi access point and server in a place like a university, hospital or office park. Think of it as the road system of the building: data travels along these roads to get from one place to another.

Roads sometimes break — a cable is unplugged, a device overheats, a port dies. When a road on a normal network breaks, the network has to notice the break, figure out a new route, and tell everyone about it. While that is happening, traffic piling up at the break is simply lost. For the people using it, that shows up as a frozen video call, a failed payment, a dropped login — a small outage every time something fails.

The goal of this project is simple to state: when any single cable or device fails, the network should heal itself so fast that users barely notice, and no traffic should be permanently lost.

# 2. How networks normally handle failures (and why it's not great)

Traditional networks have no central boss. Every switch is on its own and runs a distributed protocol — the switches gossip with each other to agree on routes. The common ones are called STP/RSTP (for simple local networks) and OSPF, IS-IS and BGP (for routing between larger areas).

This gossip approach has two real drawbacks:

- It can be slow. After a failure the switches must detect it, flood the news to each other, and each independently recompute routes. For the routing protocols (OSPF/BGP) this often takes hundreds of milliseconds to tens of seconds — long enough for calls to drop and sessions to time out.
- It wastes capacity. To avoid loops, the simple protocol (RSTP) deliberately switches off about half the links and keeps them idle as standby. You paid for those links but can't use them day to day.

In short: the network does eventually recover, but slowly and inefficiently, because no one has the full picture.

# 3. Our approach: give the network a brain (SDN)

Software-Defined Networking (SDN) splits the network into two parts, like an air-traffic-control tower and the planes:

- The controller (the tower) is software with a complete, live map of every switch and cable. It makes all the routing decisions.
- The switches (the planes) just follow the instructions they're given — they forward traffic along rules the controller installs in advance.

Because one brain sees everything, it can plan smarter routes using all the links, and when something breaks it can react in a coordinated, fast way instead of waiting for every switch to gossip and agree.

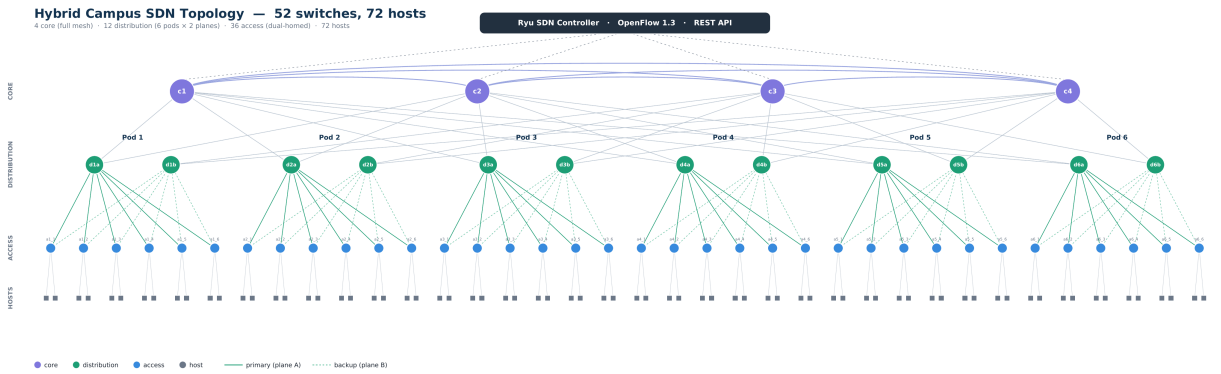
We built this using industry-standard open tools: OpenFlow 1.3 (the language the controller uses to program switches), the Ryu controller, Open vSwitch software switches, and Mininet to emulate a whole campus on one laptop.

# 4. What we actually built

We designed and built a realistic campus network with 52 switches and 72 host computers, arranged in the standard three layers used in real data-centre and campus design:

- Core (4 switches) — the high-speed centre, all cross-connected.
- Distribution (12 switches) — two per building-zone ('pod'), for redundancy.
- Access (36 switches) — what the 72 host computers plug into.

The key design idea is two independent paths for everything. We built the network as two parallel planes (A and B) that share nothing in the middle, and every access switch is wired to both of its distribution switches ('dual-homed'). So between any two computers there are always two completely separate routes — if one is hit by a failure, the other is untouched.




*The full network we built: 4 core (purple), 12 distribution (green), 36 access (blue) switches and 72 hosts, with two independent planes.*

## Two safety mechanisms

We combined two layers of protection, each suited to a different kind of failure:

- Pre-planned local backups (OpenFlow fast-failover groups). Every switch is given a primary and a backup exit in advance, so for simple link failures it can switch over instantly on its own — like a driver who already knows the detour before the road closes.
- Central re-routing by the controller. For bigger failures (a whole switch dying), the controller — with its full map — recomputes fresh routes around the damage and installs them, in well under a tenth of a second.

# 5. How we proved it works (no hand-waving)

A dashboard turning a line green is not proof. So we built an automated test harness that injects real failures into the running network (it actually shuts ports down, exactly like a real cable fault) while traffic is flowing, and measures what happens — over 103 randomized trials across five different failure types.

For every trial we measured, with real packet traces and the controller's own logs: how long the outage lasted, how many packets were lost, how long recovery took, and

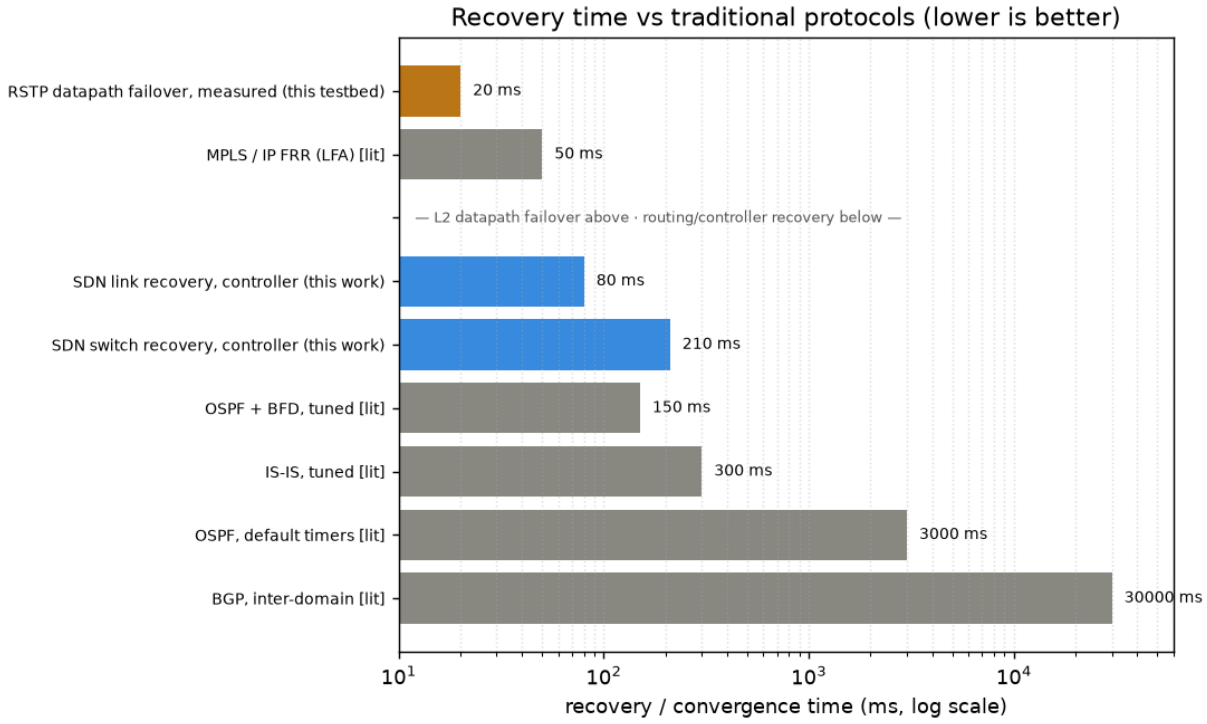
whether connectivity fully came back. We report medians and spreads, not cherry-picked single runs.

## 6. The results

| What failed | How we recovered | Outage | Traffic restored |
|---|---|---|---|
| A link (cable) | Rerouted | ~80 ms | 100% |
| A whole switch | Rerouted | ~210 ms | 100% |
| The core mesh | Survived on remaining links | ~0 ms | 100% |

Every single one of the 103 failures recovered with 100% of traffic restored, and the network kept running the whole time. The control 'brain' was cheap to run: about 5.8% CPU and 65 MB of memory, managing 2238 forwarding rules across the 52 switches.

Here is how our recovery compares with the traditional protocols:




*Recovery time (lower is better, log scale). Our controller recovery (~80–210 ms) is far faster than OSPF or BGP, the protocols a normal campus would use for routing.*

## 7. What we found honestly (and its limits)

We tested everything on emulated switches running on a laptop, which is standard for research but differs from real hardware in one way worth stating: the instant 'local backup' switch-over is a hardware feature that the software switches don't fully perform. We verified this directly by inspecting the switches during a failure (and even tried the standard 'BFD' fix).

So in our tests the recovery is driven by the controller rather than the local backup — and it is still sub-100 ms for links and ~210 ms for a whole switch, with 100% delivery. On real hardware the pre-planned local backups would add instant (sub-50 ms) protection on top. Being upfront about this is part of doing the measurement properly.

## 8. Why this is better than the normal model

- Much faster than routing protocols. Against OSPF/IS-IS/BGP — the real alternative for campus routing — our recovery is 10 to 1000 times faster.
- Uses every link. The traditional simple protocol (RSTP) switches off ~half the links as idle standby; our design forwards on all of them with two live, independent paths.
- One brain, smarter decisions. The controller has a global, live map, so it picks better routes and can enforce policy — impossible when every switch decides alone.
- Handles device failures gracefully. A whole switch dying is recomputed and routed around automatically, with no permanent loss.
- Measured, not claimed. Every number here comes from real injected failures with packet-level evidence, repeated many times.

## 9. In one sentence

*We built a campus network with a central software brain and two independent paths for everything, and showed — with real, repeated experiments — that it heals from cable and device failures in well under a tenth of a second with zero traffic lost, far faster and more efficiently than the protocols networks use today.*